

Competitive Programming

Section 35 Editorial: Math Mixed Contest

Competitive Programming Course

2026-07-10

Table of contents

1 Editorial overview	3
1.1 Read this derivation before the code	3
2 A. Blocked Path	4
2.1 Algorithm	4
2.2 Correctness proof	4
2.3 Full C++ solution	4
2.4 Full Python solution	5
3 B. Coprime Pairs	7
3.1 Algorithm	7
3.2 Correctness proof	7
3.3 Full Python solution	7
3.4 Full C++ solution	8
4 C. At Least One	10
4.1 Algorithm	10
4.2 Correctness proof	10
4.3 Full C++ solution	10
4.4 Full Python solution	11
5 Testing discussion	12
6 What to carry forward	13
7 Problem-local upsolve checklist	14
7.1 A. Blocked Path	14
7.2 B. Coprime Pairs	14
7.3 C. At Least One	15

1 Editorial overview

1.1 Read this derivation before the code

The detailed upsolve material for each problem is collected in **Problem-local upsolve checklist** below the compact reference listings. Read the corresponding block before studying its implementation:

Problem	State/model to establish	Trace to perform	Main failure mode
A	all monotone paths minus paths through one block	3 by 4, block (2,2)	applying the formula to multiple blocks
B	seen [d] for square-free divisors	process 1, 6, 10, 21, 30	updating seen before counting
C	product of no-event probabilities	1/2, 1/3, 1/4	multiplying without independence

The source listings are intentionally compact reference implementations. The checklist supplies the constraints, pseudocode, proof checkpoints, complexity, language notes, adversarial tests, and brute-force oracle to use before code.

Problem	Main tools
A. Blocked Path	combinations, subtract bad paths
B. Coprime Pairs	SPF, distinct prime factors, inclusion-exclusion
C. At Least One	complement probability

2 A. Blocked Path

2.1 Algorithm

```
total = C(H+W-2, H-1)
bad = C(R+C-2, R-1) * C(H-R+W-C, H-R)
answer = total - bad
```

2.2 Correctness proof

Every monotone path is determined by the positions of its down moves. The bad paths are exactly those that can be split into a monotone path from the start to the blocked cell and then from the blocked cell to the finish. These two parts are independent, so multiplication counts them. Subtracting all bad paths leaves exactly the valid paths. $\square$

2.3 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long r = 1;
    while (b) {
        if (b & 1LL) r = r * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return r;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int h, w, r, c;
    if (!(cin >> h >> w >> r >> c)) return 0;
    int limit = h + w;
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[limit] = mod_pow(fact[limit], MOD - 2);
```

```

for (int i = limit; i >= 1; i--) invfact[i - 1] = invfact[i] * i % MOD;
auto C = [&](int n, int k) -> long long {
    if (k < 0 || k > n) return 0;
    return fact[n] * invfact[k] % MOD * invfact[n - k] % MOD;
};
long long total = C(h + w - 2, h - 1);
long long through = C(r + c - 2, r - 1) * C(h - r + w - c, h - r) % MOD;
cout << (total - through + MOD) % MOD << '\n';
return 0;
}

```

2.4 Full Python solution

```

import sys

MOD = 1_000_000_007

def prepare(n: int) -> tuple[list[int], list[int]]:
    fact = [1] * (n + 1)
    for i in range(1, n + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (n + 1)
    invfact[n] = pow(fact[n], MOD - 2, MOD)
    for i in range(n, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact

def comb(n: int, k: int, fact: list[int], invfact: list[int]) -> int:
    if k < 0 or k > n:
        return 0
    return fact[n] * invfact[k] % MOD * invfact[n - k] % MOD

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    h, w, r, c = data
    fact, invfact = prepare(h + w)
    total = comb(h + w - 2, h - 1, fact, invfact)
    through = (
        comb(r + c - 2, r - 1, fact, invfact)
        * comb(h - r + w - c, h - r, fact, invfact)
        % MOD
    )

```

```
    print((total - through) % MOD)

if __name__ == "__main__":
    main()
```

3 B. Coprime Pairs

3.1 Algorithm

Process numbers left to right. For current x , get its distinct prime factors. Use inclusion-exclusion over nonempty subsets to count previous numbers sharing at least one factor. Add $\text{previous_count} - \text{shared}$ to the answer, then update all subset-product counters for x .

3.2 Correctness proof

For current x , a previous number is non-coprime with x iff it is divisible by at least one distinct prime factor of x . The subset-product counters give the size of every intersection of these divisibility events. Inclusion-exclusion therefore gives the number of previous non-coprime values. All other previous values form coprime pairs with x . Each pair is counted when its second endpoint is processed. $\square$

3.3 Full Python solution

```
import sys
from collections import defaultdict

def spf_table(n: int) -> list[int]:
    spf = list(range(n + 1))
    if n >= 1:
        spf[1] = 1
    for p in range(2, n + 1):
        if spf[p] == p and p * p <= n:
            for x in range(p * p, n + 1, p):
                if spf[x] == x:
                    spf[x] = p
    return spf

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    arr = data[1 : 1 + n]
    spf = spf_table(max(arr))
    seen = defaultdict(int)
    answer = 0
```

```

for idx, x in enumerate(arr):
    factors = []
    y = x
    while y > 1:
        p = spf[y]
        factors.append(p)
        while y % p == 0:
            y //= p
    share = 0
    for mask in range(1, 1 << len(factors)):
        prod = 1
        bits = 0
        for i, p in enumerate(factors):
            if (mask >> i) & 1:
                prod *= p
                bits += 1
        share += seen[prod] if bits % 2 else -seen[prod]
    answer += idx - share
    for mask in range(1, 1 << len(factors)):
        prod = 1
        for i, p in enumerate(factors):
            if (mask >> i) & 1:
                prod *= p
        seen[prod] += 1
print(answer)

```

3.4 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<int> a(n);
    int mx = 1;
    for (int &x : a) {
        cin >> x;
        mx = max(mx, x);
    }

    vector<int> spf(mx + 1);
    iota(spf.begin(), spf.end(), 0);
    if (mx >= 1) spf[1] = 1;
    for (long long p = 2; p <= mx; p++) {

```

```

        if (spf[p] == p && p * p <= mx) {
            for (long long x = p * p; x <= mx; x += p) {
                if (spf[x] == x) spf[x] = p;
            }
        }
    }

    vector<int> seen(mx + 1, 0);
    long long answer = 0;
    for (int idx = 0; idx < n; idx++) {
        int x = a[idx];
        vector<int> factors;
        while (x > 1) {
            int p = spf[x];
            factors.push_back(p);
            while (x % p == 0) x /= p;
        }

        long long share = 0;
        int m = (int)factors.size();
        for (int mask = 1; mask < (1 << m); mask++) {
            int prod = 1, bits = 0;
            for (int i = 0; i < m; i++) {
                if (mask & (1 << i)) {
                    prod *= factors[i];
                    bits++;
                }
            }
            if (bits % 2) share += seen[prod];
            else share -= seen[prod];
        }
        answer += idx - share;

        for (int mask = 1; mask < (1 << m); mask++) {
            int prod = 1;
            for (int i = 0; i < m; i++) {
                if (mask & (1 << i)) prod *= factors[i];
            }
            seen[prod]++;
        }
    }

    cout << answer << '\n';
    return 0;
}

```

4 C. At Least One

4.1 Algorithm

Compute the probability that no event happens, then subtract from 1:

```
none = product((q_i-p_i)/q_i)
answer = 1 - none
```

4.2 Correctness proof

The complement of “at least one event happens” is “no event happens.” Events are independent, so the probability that none happen is the product of the individual failure probabilities. Complementing gives the desired probability. $\square$

4.3 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long r = 1;
    while (b) {
        if (b & 1LL) r = r * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return r;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    long long none = 1;
    for (int i = 0; i < n; i++) {
        long long p, q;
        cin >> p >> q;
        none = none * ((q - p) % MOD) % MOD * mod_pow(q, MOD - 2) % MOD;
    }
}
```

```

}
cout << (1 - none + MOD) % MOD << '\n';
return 0;
}

```

4.4 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    idx = 1
    none = 1
    for _ in range(n):
        p, q = data[idx], data[idx + 1]
        idx += 2
        none = none * ((q - p) % MOD) % MOD * pow(q, MOD - 2, MOD) % MOD
    print((1 - none) % MOD)

if __name__ == "__main__":
    main()

```

5 Testing discussion

The mixed contest generators compare against direct combinatorics, brute-force gcd pair checks, and exact rational probability. This phase is especially prone to off-by-one and complement mistakes, so stress tests are part of the workflow.

6 What to carry forward

Math contest problems are often a composition of small tools. The main skill is recognizing the composition early: combinations plus subtraction, sieve plus inclusion-exclusion, or probability plus complement. # Extended upsolve workshop

A. Blocked Path. Subtract $\text{ways}(\text{start}, \text{block}) * \text{ways}(\text{block}, \text{finish})$ from all monotone paths. Test a blocked start or finish, where every path is forbidden.

B. Coprime Pairs. For each value, inclusion-exclusion over its distinct prime factors counts earlier values sharing any factor; subtract from prior count. Test value one and repeated composite values.

C. At Least One. Compute $1 - \text{product}(1 - p_i/q_i)$. Directly adding event probabilities double-counts overlapping outcomes; test one certain event. # Full upsolve notes

A. Blocked Path. Every forbidden monotone path has a unique prefix ending at (R, C) and suffix from it, so subtract $C(R+C-2, R-1) * C(H-R+W-C, H-R)$ from all paths. For a 3-by-3 grid blocked in the middle: $6 - 2 * 2 = 2$. Test a blocked start/end, where the answer is zero.

B. Coprime Pairs. Maintain $\text{seen}[d]$, the number of prior values divisible by square-free d . For 30 after $[6, 10, 21]$, inclusion-exclusion gives $\text{seen}[2] + \text{seen}[3] + \text{seen}[5] - \text{seen}[6] - \text{seen}[10] - \text{seen}[15] + \text{seen}[30]$ all earlier values share a factor. Count before updating the subset products. SPF plus at most 127 subsets per local value gives safe complexity. Test one, duplicates, and a three-factor value.

C. At Least One. Compute $\text{none} = \text{product}((q-p) * \text{inverse}(q))$, then answer $1 - \text{none}$. For $1/2$ and $1/3$, this is $1 - 1/2 * 2/3 = 2/3$. Independence justifies the product; a certain event and a zero-probability event are essential tests.

7 Problem-local upsolve checklist

7.1 A. Blocked Path

7.1.1 Constraints and model

H and W are up to one million, so grid DP over $H \times W$ cells is not viable. There is exactly one blocked cell. A monotone path is completely determined by the positions of its down moves, which makes binomial coefficients the correct model. Precompute factorials through $H+W$ once.

7.1.2 Trace and pseudocode

For $H=3, W=4, R=2, C=2$, all paths are $C(5,2)=10$. The blocked prefix count is $C(2,1)=2$; the suffix count is $C(3,1)=3$; subtract 6 to get 4.

```
all = choose(H+W-2, H-1)
through = choose(R+C-2, R-1) * choose(H-R+W-C, H-R)
answer = all - through modulo MOD
```

Every forbidden path has one and only one such prefix/suffix pair, proving the multiplication and subtraction. The time is $O(H+W)$ preprocessing and $O(1)$ work afterward. In C++ use `long long` products; Python uses the same factorial lists. Wrong answers usually forget that a blocked start or end makes the answer zero, or apply this one-block formula to multiple blocks. Test both endpoints, a one-row grid, and a block adjacent to an endpoint.

7.2 B. Coprime Pairs

7.2.1 Constraints and state

The array has up to 200,000 values, each at most one million. Build SPF once. For each square-free divisor d , `seen[d]` is the number of already processed values divisible by d . Prime powers are deliberately absent: sharing a prime is the event being counted.

7.2.2 Trace and pseudocode

When processing $30=2 \cdot 3 \cdot 5$ after $[6, 10, 21]$, count shared values as:

```
+seen[2] + seen[3] + seen[5]
-seen[6] - seen[10] - seen[15]
+seen[30]
```

This is 3, hence it contributes $3-3=0$ coprime pairs. Then increment every one of those seven seen entries for future values.

```
factors = distinct prime factors of x from SPF
shared = alternating sum seen[product(subset)]
answer += processed - shared
for every nonempty subset: seen[product(subset)] += 1
```

An earlier value is included in the alternating sum once iff it shares at least one prime with x ; otherwise it appears in no selected event. Therefore `processed-shared` is precisely the number of new coprime pairs. With at most seven distinct factors, each value creates at most 127 subsets. Complexity is $O(M \log \log M + \sum 2^{\omega(a[i])})$ time and $O(M)$ memory. Test 1 (no subsets, coprime with all prior values), repeated composites, and primes.

7.2.3 Implementation and failure modes

Generate subset products from distinct factors, not from the factorization with multiplicity. Count before updating `seen`. Use `long long` for the answer: there can be about $n*(n-1)/2$ pairs. A brute-force oracle for random arrays of length at most 10 should compare every gcd directly.

7.3 C. At Least One

7.3.1 Constraints and model

The statement explicitly gives independent events and prime-modulus rational probabilities with nonzero denominators. Directly adding event probabilities overcounts outcomes where several events occur. The complement has one product: none of the events happens.

7.3.2 Trace, proof, and code shape

For probabilities $1/2$, $1/3$, $1/4$, the no-event probability is $(1/2)*(2/3)*(3/4)=1/4$, so the answer is $3/4$. In modular arithmetic compute each factor as $(q-p) * \text{inverse}(q)$, multiply them, and normalize $1-\text{none}$.

```
none = 1
for p, q:
    none = none * (q-p) * inverse(q) modulo MOD
print (1-none) modulo MOD
```

Independence proves the product; complementarity proves the final subtraction. The algorithm takes $O(n \log \text{MOD})$ with Fermat inverses and $O(1)$ extra memory. Test one event, a zero-probability event, and a certain event. Do not use this method if events are dependent or a denominator is zero modulo MOD.